



**POLITÉCNICO
ESTELLA**

**ASIGNATURA: PROGRAMACIÓN DE SERVICIOS Y
PROCESOS**

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

PSP06 TAREA 6

Presentado por: Eugen Moga

Fecha: 09/03/2026

ÍNDICE

Apartado 1 1

Apartado 2..... 3

Probando el funcionamiento..... 5

Apartado 1

Creo el proyecto PSP06 con la clase principal y dentro del método main declaro una variable de tipo Scanner para leer la entrada por teclado del usuario.

```
19 public static void main(String[] args) {  
20  
21     // Scanner para leer entradas del teclado  
    Scanner sc = new Scanner(System.in);
```

Con System.out muestro en pantalla el mensaje de: Apartado 1 y que inserte * para terminar el bucle del usuario.

Declaro la variable usuario de tipo String para guardar la entrada del usuario.

```
25  
26 // APARTADO 1: Bucle de validación de usuario  
27 System.out.println("APARTADO1: Validación Nombre de usuario");  
28 System.out.println("{Introduce '*' para terminar el bucle para el nombre de usuario}\n");  
29  
30 // Declaro la variable fuera del bucle para poder  
31 // reutilizarla en la condición del while  
32 String usuario;
```

Dentro de un bucle while(true) mientras sea verdadero solicito al usuario que inserte un nombre de usuario y lo guardo en la variable usuario

```
34 // El bucle se repite indefinidamente hasta que el usuario  
35 while (true) {  
36  
37     System.out.print("Introduce el nombre de usuario: ");  
38     usuario = sc.nextLine();
```

La primera comprobación que hago es para comprobar si el usuario ha insertado (*) para finalizar el bucle. Si se cumple muestro el mensaje fin de la entrada de usuarios y salgo del bucle con break;

```
39  
40 // Condición de salida: si es '*', terminamos el bucle  
41 if (usuario.equals("*")) {  
42     System.out.println("Fin de la entrada de usuarios.\n");  
43     break;  
44 }
```

Si de momento la primera comprobación no se cumple hago la segunda comprobación.

Compruebo si el usuario no cumple con el patrón de validación. Para ello utilizo las expresiones regulares de `String.matches()`.

Donde le indico que debe empezar con un dígito en minúscula `[a-z]` y que debe seguir con letras minúsculas o dígitos `[a-z0-9]` durante un mínimo de 3 a 7 caracteres `{3,7}`. Esto es debido a que la longitud total es de 4 a 8 caracteres, pero como empiezo con un carácter de letra minúscula le resto uno al total.

Muestro el mensaje de error.

Si la cadena se cumple muestro el mensaje usuario válido.

```
45
46 // matches() devuelve true si la cadena cumple el patrón
47 // Si NO cumple (!) muestro el error
48 if (!usuario.matches("[a-z][a-z0-9]{3,7}")) {
49     System.out.println("El nombre de usuario no es válido.");
50     System.out.println("Recuerda: 4-8 caracteres, empieza por minúscula,");
51     System.out.println("solo letras minúsculas y números.\n");
52 } else {
53     System.out.println("Usuario " + usuario + " válido.\n");
54 }
55 }
```

Esto se repite en bucle hasta que el usuario inserta *. En ese momento se sale del bucle para validar usuarios y sigue la ejecución del programa llegando al apartado 2

Apartado 2

Muestro por pantalla los requisitos para validar un producto y declaro la variable producto de tipo String para guardar la entrada del usuario.

```
56
57 // APARTADO 2: Bucle de validación de código de producto
58 System.out.println("Validacion de codigo de producto");
59 System.out.println("Formato esperado: ZZZ-CC-CODIG-ADVE");
60 System.out.println("ZZZ : 3 dígitos binarios (ej: 010)");
61 System.out.println("CC : categoría TM/OT/ON/OC/RI/ES/EN");
62 System.out.println("CODIG: 5 dígitos numéricos");
63 System.out.println("ADVE : letra B/C/D + 3 dígitos (ej: B290)");
64 System.out.println("(Introduce '*' para terminar)\n");
65
66 String producto;
```

Dentro de un bucle while(true) mientras se cumpla. Solicito el usuario que inserte el código del producto para validar. Guardo la entrada del usuario en la variable producto.

```
67
68 while (true) {
69
70     System.out.print("Introduce el código de producto: ");
71     producto = sc.nextLine();
72 }
```

Primero compruebo si el usuario ha insertado (*) si se cumple muestro un mensaje fin de la entrada de productos y salgo del bucle con break.

```
72
73 // Condición de salida
74 if (producto.equals("*")) {
75     System.out.println("Fin de la entrada de productos.");
76     break;
77 }
```

Igual que en el apartado 1 utilizo String.matches() para pasarle el patrón de validación. El código del producto tiene que empezar con 3 dígitos binario [01]{3} seguido de un guion –

La categoría la pongo así: (TM|OT|ON|OC|RI|ES|EN) seguido de un guion –

5 dígitos numéricos [0-9]{5} seguido de un guion –

La letra B, C o D seguida de 3 dígitos [BCD][0-9]{3}

Toda esta cadena en conjunto sería:

[01]{3}-(TM|OT|ON|OC|RI|ES|EN)-[0-9]{5}-[BCD][0-9]{3}

Si no cumple muestro un mensaje de error y enseño un ejemplo válido. Y se sigue repitiendo el bucle.

Si cumple muestro por pantalla el mensaje Código válido. Y sigue repitiendo el bucle hasta que el usuario inserta (*).

```
78 |
79 | if (!producto.matches("[01]{3}-(TM|OT|ON|OC|RI|ES|EN)-[0-9]{5}-[BCD][0-9]{3}")) {
80 |     System.out.println("Error el código de producto no tiene el formato correcto.");
81 |     System.out.println("Ejemplo válido: 010-TM-12345-B290\n");
82 | } else {
83 |     System.out.println("Código " + producto + " válido.\n");
84 | }
85 |
```

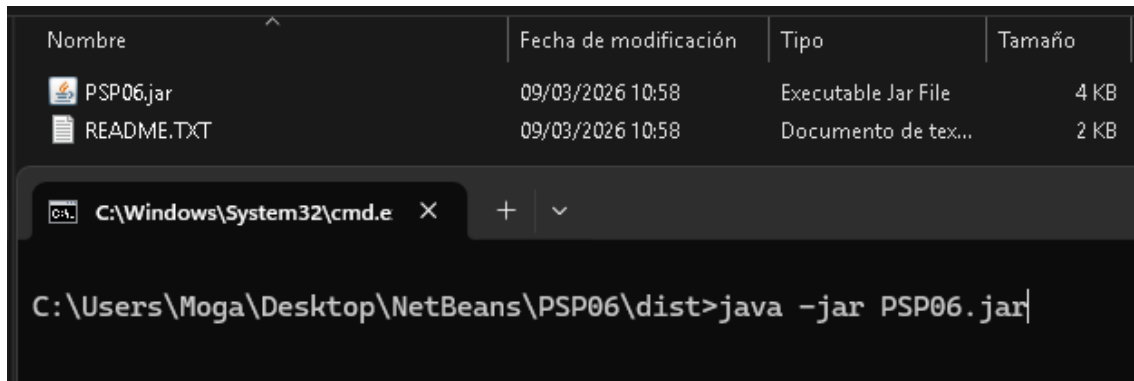
Cuando el usuario inserta un (*) se sale del bucle del segundo apartado cierro el scanner y muestro el mensaje programa finalizado.

```
86 |
87 | // Cierro el Scanner al terminar (buena práctica)
88 | sc.close();
89 | System.out.println("\nPrograma finalizado.");
90 |
```

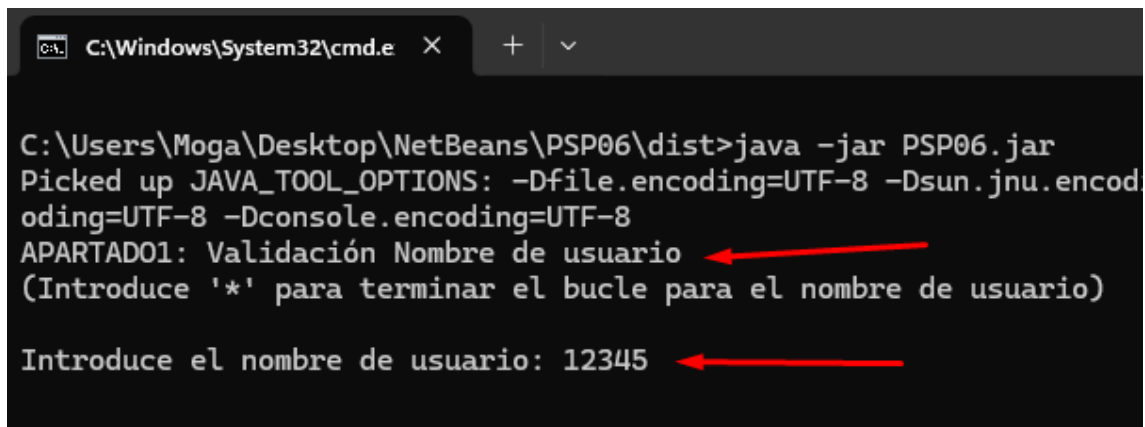
Probando el funcionamiento

Compilo el proyecto, voy a la carpeta dist del proyecto donde me ha generado el archivo PSP06.jar abro la terminal y ejecuto el archivo con el comando:

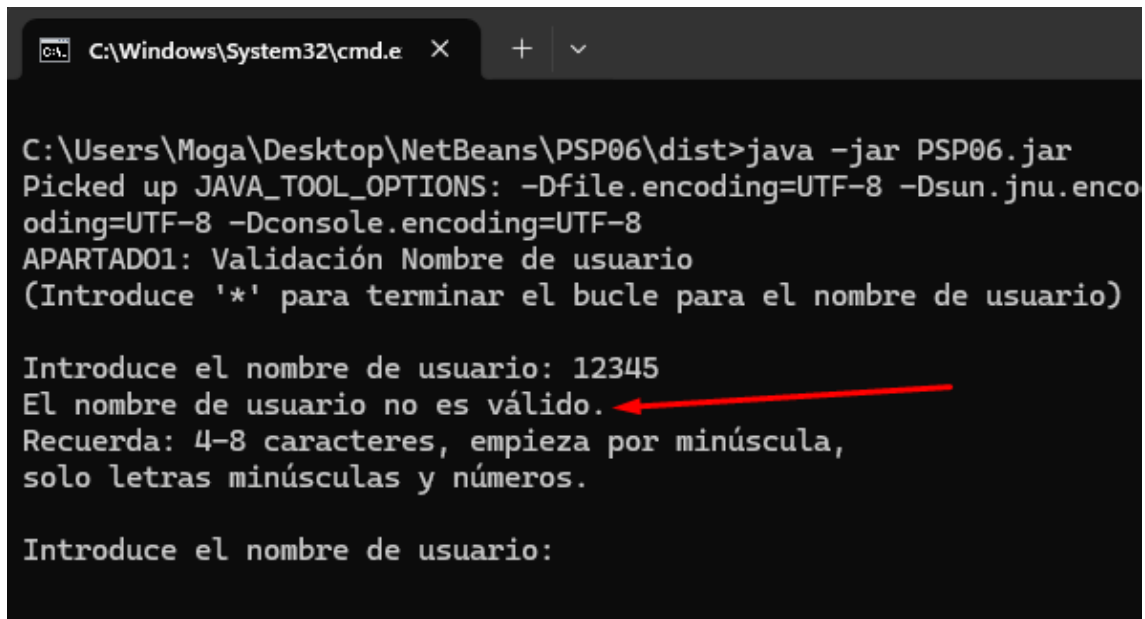
Java -jar PSP06.jar



Me muestra el mensaje APARTADO 1 y me solicita que introduzca el nombre de usuario.



En este caso como he insertado un nombre no valido me muestra el mensaje de error.

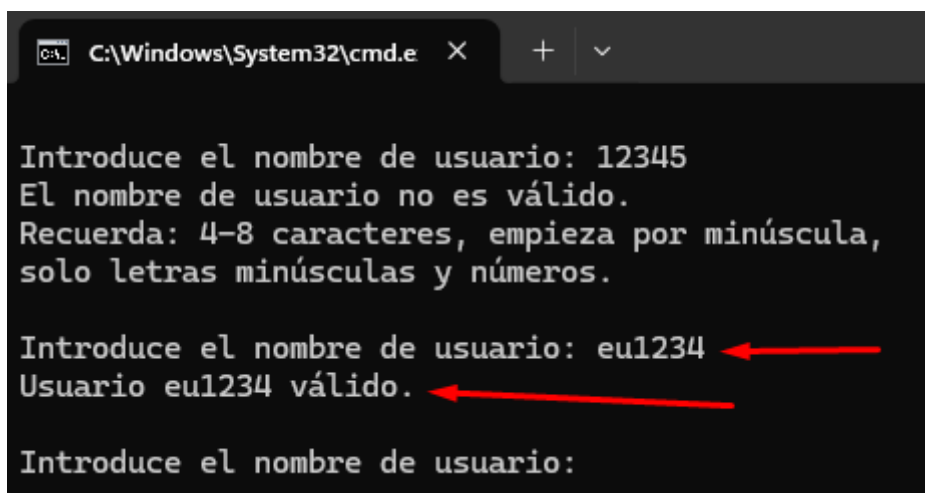


```
C:\Users\Moga\Desktop\NetBeans\PSP06\dist>java -jar PSP06.jar
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8
APARTADO1: Validación Nombre de usuario
(Introduce '*' para terminar el bucle para el nombre de usuario)

Introduce el nombre de usuario: 12345
El nombre de usuario no es válido.
Recuerda: 4-8 caracteres, empieza por minúscula,
solo letras minúsculas y números.

Introduce el nombre de usuario:
```

Al insertar un nombre de usuario válido me muestra el mensaje: Usuario válido.

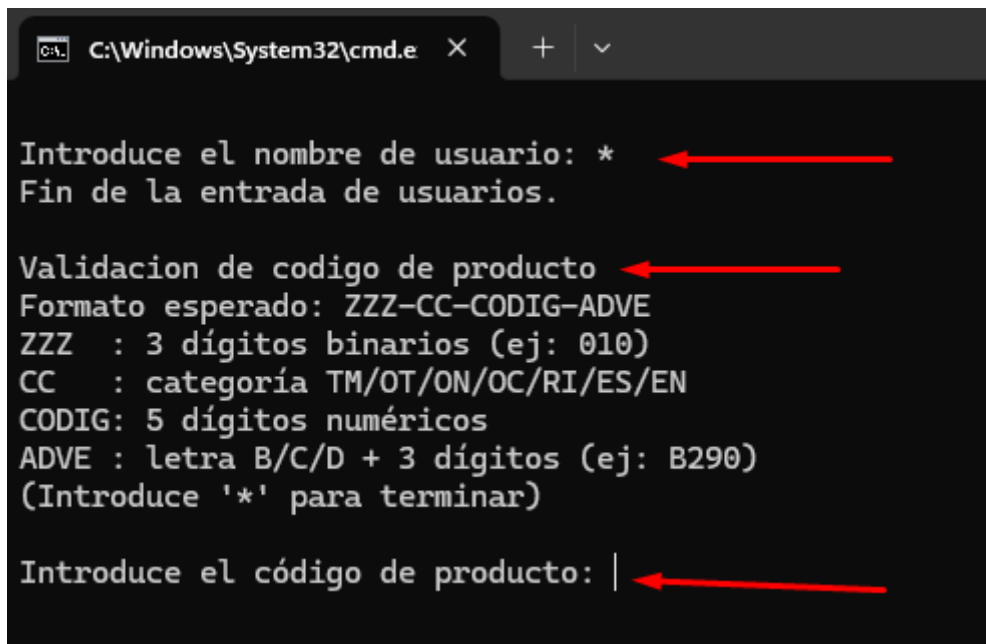


```
Introduce el nombre de usuario: 12345
El nombre de usuario no es válido.
Recuerda: 4-8 caracteres, empieza por minúscula,
solo letras minúsculas y números.

Introduce el nombre de usuario: eu1234
Usuario eu1234 válido.

Introduce el nombre de usuario:
```


Inserto el asterisco para salir del bucle y pasar al apartado 2



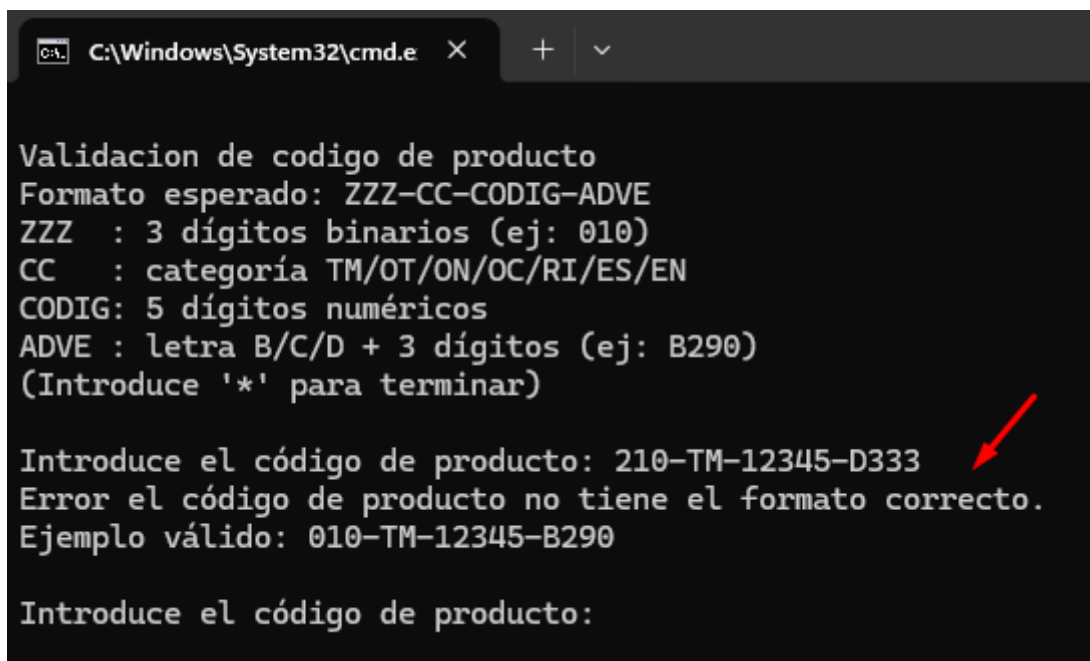
```
C:\Windows\System32\cmd.e X + v

Introduce el nombre de usuario: *
Fin de la entrada de usuarios.

Validacion de codigo de producto
Formato esperado: ZZZ-CC-CODIG-ADVE
ZZZ : 3 dígitos binarios (ej: 010)
CC : categoría TM/OT/ON/OC/RI/ES/EN
CODIG: 5 dígitos numéricos
ADVE : letra B/C/D + 3 dígitos (ej: B290)
(Introduce '*' para terminar)

Introduce el código de producto: |
```

Introduzco un código no valido y me muestra el mensaje de error porque el código no tiene el formato correcto



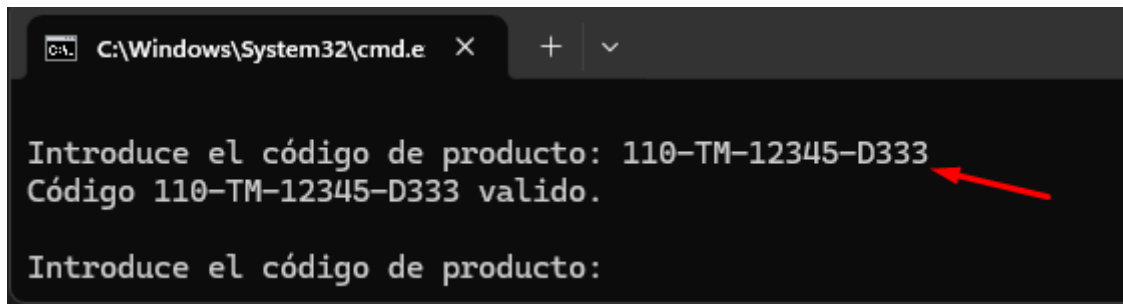
```
C:\Windows\System32\cmd.e X + v

Validacion de codigo de producto
Formato esperado: ZZZ-CC-CODIG-ADVE
ZZZ : 3 dígitos binarios (ej: 010)
CC : categoría TM/OT/ON/OC/RI/ES/EN
CODIG: 5 dígitos numéricos
ADVE : letra B/C/D + 3 dígitos (ej: B290)
(Introduce '*' para terminar)

Introduce el código de producto: 210-TM-12345-D333
Error el código de producto no tiene el formato correcto.
Ejemplo válido: 010-TM-12345-B290

Introduce el código de producto:
```

Al insertar el código correctamente me muestra el mensaje que el código es válido.



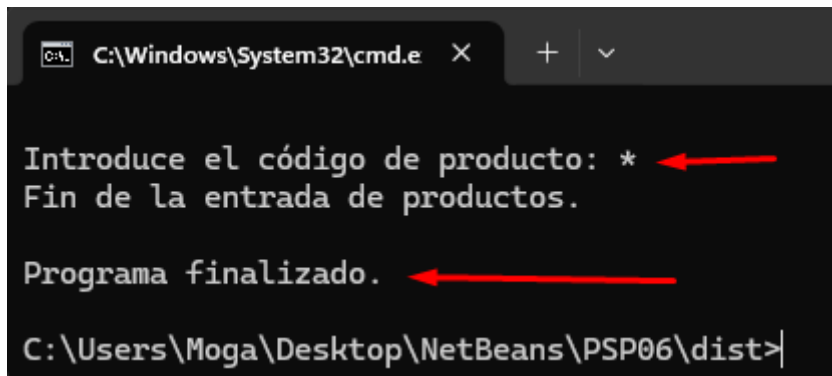
```
C:\Windows\System32\cmd.e X + v

Introduce el código de producto: 110-TM-12345-D333
Código 110-TM-12345-D333 valido.

Introduce el código de producto:
```

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32\cmd.e' and standard window controls. The command prompt displays a prompt 'Introduce el código de producto:' followed by the input '110-TM-12345-D333'. The next line shows the response 'Código 110-TM-12345-D333 valido.' with a red arrow pointing to the word 'valido.'. The prompt 'Introduce el código de producto:' is shown again on the next line.

Inserto * para finalizar el programa



```
C:\Windows\System32\cmd.e X + v

Introduce el código de producto: *
Fin de la entrada de productos.

Programa finalizado.

C:\Users\Moga\Desktop\NetBeans\PSP06\dist>
```

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32\cmd.e' and standard window controls. The command prompt displays a prompt 'Introduce el código de producto:' followed by the input '*'. The next line shows the response 'Fin de la entrada de productos.' with a red arrow pointing to the asterisk. The following line shows 'Programa finalizado.' with a red arrow pointing to the end of the sentence. The final line shows the command prompt path 'C:\Users\Moga\Desktop\NetBeans\PSP06\dist>'.